Write a mongo DB program to create a student collection having fields:

Ans:

```
from pymongo import MongoClient

# Connect to MongoDB (default localhost:27017)
client = MongoClient("mongodb://localhost:27017/")
db = client["college"]
student_collection = db["student"]

# 1. Insert 5 documents into Student collection
students = [
    {"RollNo": 1, "Name": "Alice", "Course": "MCA", "Marks": 85, "GradePoint": 9},
    {"RollNo": 2, "Name": "Bob", "Course": "MBA", "Marks": 78, "GradePoint": 7},
    {"RollNo": 3, "Name": "Charlie", "Course": "MCA", "Marks": 90, "GradePoint": 10},
    {"RollNo": 4, "Name": "David", "Course": "BCA", "Marks": 88, "GradePoint": 8},
    {"RollNo": 5, "Name": "Eve", "Course": "BBA", "Marks": 92, "GradePoint": 10}
]

student_collection.insert_many(students)
print("Inserted 5 student records.\n")

# 2. Find students having marks between 80 and 90
print("Students with marks between 80 and 90:")
for student in student_collection.find({"Marks": {"$gte": 80, "$lte": 90}}):
    print(student)

# 3. Update name of a student whose roll no. is 5
student_collection.update_one({"RollNo": 5}, {"$set": {"Name": "Eva"}})
print("\nUpdated name of student with RollNo 5.\n")

# 4. Display top 3 students according to their grade points
```

```
print("Top 3 students by grade point:")
for student in student_collection.find().sort("GradePoint", -1).limit(3):
    print(student)

# 5. Display students having highest grade points
max_grade = student_collection.find_one(sort=[("GradePoint", -1)])["GradePoint"]
print("\nStudents with highest grade point:")
for student in student_collection.find({"GradePoint": max_grade}):
    print(student)

# 6. Find all students having course 'MCA'
print("\nStudents enrolled in MCA:")
for student in student_collection.find({"Course": "MCA"}):
    print(student)

# 7. Display all students in descending order of marks
print("\nAll students in descending order of marks:")
for student in student_collection.find().sort("Marks", -1):
    print(student)
```

Q. write a code to send date and time from views py to template file

Ans:

- Modify View

```
from django.shortcuts import render
from datetime import datetime

def show_datetime(request):
    current_datetime = datetime.now()
    return render(request, 'datetime.html', {'current_datetime': current_datetime})
```

- Create Template

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Date and Time</title>
</head>
<body>
  <h1>Current Date and Time</h1>
  <p>{{ current_datetime }}</p>
</body>
</html>
```

- Add URL

```
from django.urls import path
from .views import show_datetime

urlpatterns = [
  path('datetime/', show_datetime, name='show_datetime'),
]
```

Q. write short note on django rest framework in python

Ans:

Key Features:

- **Easy Serialization**: Converts Django models and querysets into JSON, XML, etc., using serializers.
- **Class-Based Views**: Provides generic views for common patterns like list, create, update, and delete.
- **Authentication & Permissions**: Built-in support for user authentication, permissions, and throttling.
- **Browsable API**: Auto-generates a clean, interactive web UI for testing your APIs.
- **Pagination, Filtering, and Versioning**: Easily add pagination, filter queries, and manage API versions.
- **Third-Party Integration**: Works well with tools like Swagger (for API docs) and JWT (for authentication).

🔷 **Use Cases:**

- Creating RESTful APIs for mobile or frontend apps.

- Building backend services for web apps.
- Integrating third-party systems using HTTP APIs.

Example:

```
from rest_framework import serializers, viewsets
from myapp.models import Book


class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = '__all__'


class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Q. Design django web page for student registration to a course and display it.

Ans:

- Models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    course = models.CharField(max_length=100)

    def __str__(self):
        return f"{self.name} - {self.course}"
```

- Forms

```
from django import forms
from .models import Student

class StudentRegistrationForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'email', 'course']
```

- Views.py

```
from django.shortcuts import render, redirect
from .forms import StudentRegistrationForm
from .models import Student
```

```
def register_student(request):
    if request.method == 'POST':
        form = StudentRegistrationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = StudentRegistrationForm()
    return render(request, 'register.html', {'form': form})

def student_list(request):
    students = Student.objects.all()
    return render(request, 'student_list.html', {'students': students})
```

- Templates

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
    <h1>Register for a Course</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Register</button>
    </form>
    <a href="{% url 'student_list' %}">View Registered Students</a>
</body>
</html>
```

- URL

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register_student, name='register_student'),
    path('students/', views.student_list, name='student_list'),
]
```

Q. Write a mongo DB program to create a Books collection having fields: Title, Publisher, Price.

Ans:

```
from pymongo import MongoClient


# Connect to MongoDB

client = MongoClient("mongodb://localhost:27017/")

db = client["library"]
```

```
books_collection = db["books"]

# 1. Insert 5 documents into Books collection
books = [
    {"Title": "Python", "Author": "Guido", "Publisher": "BPB", "Price": 500},
    {"Title": "Data Science", "Author": "James", "Publisher": "Pearson", "Price": 600},
    {"Title": "Machine Learning", "Author": "Andrew", "Publisher": "O'Reilly", "Price": 750},
    {"Title": "AI Basics", "Author": "John", "Publisher": "Pearson", "Price": 450},
    {"Title": "DBMS", "Author": "Navathe", "Publisher": "McGraw", "Price": 400}
]

books_collection.insert_many(books)
print("Inserted 5 books.\n")

# 2. Retrieve books whose publisher is 'pearson'
print("Books with publisher 'Pearson':")
for book in books_collection.find({"Publisher": {"$regex": "^pearson$", "$options": "i"}}):
    print(book)

# 3. Retrieve books whose price is between 400 and 600
print("\nBooks with price between 400 and 600:")
for book in books_collection.find({"Price": {"$gte": 400, "$lte": 600}}):
    print(book)

# 4. Retrieve books in descending order of price
print("\nBooks sorted by descending price:")
for book in books_collection.find().sort("Price", -1):
    print(book)

# 5. Update the price of book by 10% whose title is 'Python'
python_book = books_collection.find_one({"Title": "Python"})
```

```
if python_book:
    new_price = python_book["Price"] * 1.10
    books_collection.update_one({"Title": "Python"}, {"$set": {"Price": new_price}})
    print("\nUpdated price of 'Python' book by 10%.")

# 6. Update the title of a book whose author is 'Guido' and publisher is 'BPB'
books_collection.update_one(
    {"Author": "Guido", "Publisher": "BPB"},
    {"$set": {"Title": "Advanced Python Programming"}}
)
print("Updated the title of book by Guido published by BPB.\n")
```

Q. explain concept of containership and delegation in python

Ans.

Container:

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()  # Containership

    def start_car(self):
        print("Starting car...")
        self.engine.start()

# Usage
my_car = Car()
my_car.start_car()
```

Delegation:

```
class Printer:
    def print_document(self):
        print("Printing document...")

class Office:
    def __init__(self):
        self.printer = Printer()

    def do_printing(self):
        # Delegating the work to Printer
        self.printer.print_document()

# Usage
office = Office()
office.do_printing()
```

Q. write a program to validation email address using regular expression in python

Ans.

```
import re

# Regular expression pattern for validating email
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

def validate_email(email):
    if re.match(email_pattern, email):
        print(f"✅ Valid email: {email}")
    else:
        print(f"❌ Invalid email: {email}")

# Test cases
emails = [
```

```
    "user@example.com",
    "hello.world@domain.co.in",
    "wrong-email@.com",
    "noatsymbol.com",
    "valid_email123@site.org"
]

for email in emails:
    validate_email(email)
```

Q. write a function to accept a string from user and it will return reverse each word of string

Ans.

```
def reverse_each_word(sentence):
    # Split sentence into words
    words = sentence.split()
    # Reverse each word using slicing
    reversed_words = [word[::-1] for word in words]
    # Join back into a single string
    return ' '.join(reversed_words)

# Accept input from user
user_input = input("Enter a sentence: ")
result = reverse_each_word(user_input)
print("Reversed each word:", result)
```

Q. Explain the use of any five functions from the random module with suitable example.

Ans.

```
def reverse_each_word(sentence):
    # Split sentence into words
    words = sentence.split()
    # Reverse each word using slicing
```

```
    reversed_words = [word[::-1] for word in words]
    # Join back into a single string
    return ' '.join(reversed_words)

# Accept input from user
user_input = input("Enter a sentence: ")
result = reverse_each_word(user_input)
print("Reversed each word:", result)
```